

JAVA THREADING NOTES

1. What is a Thread?

A thread is a lightweight sub-process that runs independently inside a program. Java uses threads to execute multiple tasks simultaneously.

Example: Music playing + File downloading at the same time

2. Thread Class (java.lang.Thread)

Java provides a built-in class called `Thread` inside `java.lang`. It is used to create and control threads.

- No import needed
- Automatically available in Java

3. Creating a Thread

Method 1: Extending Thread

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Running thread");  
    }  
}
```

Method 2: Implementing Runnable (Recommended)

```
class MyTask implements Runnable {  
    public void run() {  
        System.out.println("Running thread");  
    }  
}
```

4. Thread Life Cycle

- **NEW** – Thread created
- **RUNNABLE** – After start()
- **RUNNING** – Executing run()
- **BLOCKED / WAITING** – Paused state
- **TERMINATED** – Execution finished

NEW → RUNNABLE → RUNNING → TERMINATED

↳ WAITING / BLOCKED (via sleep, wait, join)

5. Important Methods

- `start()` → starts new thread
- `run()` → task logic
- `sleep(ms)` → pause thread
- `join()` → wait for thread
- `isAlive()` → check running state

6. Synchronization

Used to control access to shared resources and avoid conflicts.

```
synchronized(obj) {  
    // only one thread allowed here  
}
```

7. wait / notify

- `wait()` → release lock and pause
- `notify()` → wake one thread
- `notifyAll()` → wake all threads

Must be used inside synchronized block

8. Key Rules

- Use `start()`, NOT `run()`
- Thread order is NOT guaranteed
- Use synchronization for shared data
- Threads run independently